

Bayesian Agents

Intelligent Systems II

Mário Antunes

Universidade de Aveiro

2026

Part 1: From Certainty to Uncertainty

The Limits of Pure Logic

- **The Qualification Problem:** In logic, we need an infinite number of preconditions to guarantee an action (e.g., Bird jumps IF button_pressed AND battery_not_dead AND gravity_exists...).
- **The Laziness Problem:** It is too much work to list every exception.
- **Theoretical Ignorance:** We may not know all the rules of the environment.
- **Practical Ignorance:** Even if we know the rules, we may not have all the sensors to see the current state perfectly.
- **The Solution:** Probabilistic Reasoning.

Probability: The Degree of Belief

- **Logic:** A sentence is True, False, or Unknown.
- **Probability:** An agent has a “numerical degree of belief” between 0 and 1.
- **Analogy:** A logic agent says “It is raining.” A probabilistic agent says “I am 80% sure it is raining.”
- **Frequentist vs. Bayesian:** * *Frequentist:* Probability is the limit of long-run relative frequency.
 - *Bayesian:* Probability is a subjective representation of uncertainty based on prior knowledge and new evidence.
- **Expected Value:** An agent must weigh the probability of an outcome against its magnitude. Calculated as the sum of all possible outcomes multiplied by their probabilities.

Random Variables and Sample Space

- **Random Variable (X):** A variable that takes values from a domain (e.g., $Weather \in \{sunny, rain, cloudy\}$).
- **Sample Space (Ω):** The set of all possible outcomes.
- **Probability Distribution:** A function that assigns probabilities to all possible values of X .
 - $\sum P(X = x) = 1$.
- **Joint Probability:** $P(X, Y)$ is the probability that X and Y happen simultaneously.
- **Law of Total Probability:** Allows us to find the probability of an event by considering all possible ways it can happen: $P(A) = \sum_i P(A|B_i)P(B_i)$.

Conditional Probability: The Heart of Inference

- **Definition:** The probability of an event A given that we already know event B has occurred.
- **Formula:**

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$

- **Product Rule:** $P(A \wedge B) = P(A|B)P(B)$.
- **Analogy:** The probability that a bird survives (A) changes significantly if we know a pipe is approaching (B).

Bayes' Rule: The Agent's Update Mechanism

- **The Formula:**

$$P(Cause|Effect) = \frac{P(Effect|Cause)P(Cause)}{P(Effect)}$$

- **Terminology:**

- $P(Cause)$: **Prior** (What we knew before).
 - $P(Effect|Cause)$: **Likelihood** (How well the cause explains the effect).
 - $P(Cause|Effect)$: **Posterior** (Our updated belief).
- **Usage:** Agents use this to update their “World Model” based on noisy sensor data.

Independence and Conditional Independence

- **Independence:** A and B are independent if $P(A|B) = P(A)$. (Knowledge of B tells us nothing about A).
- **Conditional Independence:** A and B are independent *given* C if:

$$P(A, B|C) = P(A|C)P(B|C)$$

- This is the “Secret Sauce” that allows us to build large, efficient probabilistic networks.

Part 2: Bayesian Networks (Probabilistic Programming)

What is a Bayesian Network?

- A **Directed Acyclic Graph (DAG)** where:
 1. Nodes represent Random Variables.
 2. Edges represent direct influence (Causality).
 3. Each node has a **Conditional Probability Table (CPT)** describing the effect of parents on children.
- **Logic vs. Bayes Net:** A logic engine uses rules; a Bayes Net uses a “Joint Probability Distribution” factored into local tables.

Factoring the Joint Distribution

- Instead of storing a massive table of every possible combination, we use the network structure:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(x_i))$$

- **Complexity Reduction:** This turns an exponential problem into a manageable one.

Inference in Bayesian Networks

- **Goal:** Compute $P(\textit{Query}|\textit{Evidence})$.
- **Exact Inference:** Variable Elimination (summing out hidden variables).
- **Approximate Inference:** Sampling (Monte Carlo methods).
- If the agent sees “Smoke,” it calculates the probability of “Fire” by propagating evidence backward through the graph.

Part 3: Modeling Decisions (Markov & MDPs)

Markov Property: The “Forgetful” Agent

- **Definition:** The future is independent of the past, given the present.
- **First-Order Markov Process:**
$$P(S_t | S_{t-1}, S_{t-2}, \dots) = P(S_t | S_{t-1}).$$
- **Why?** It prevents the agent's memory from exploding. We only care about the *current* state to predict the next one.

Markov Decision Processes (MDP)

- An MDP is a Markov Process with **Actions** and **Rewards**.
- **Components:**
 - **States (S):** Possible situations.
 - **Actions (A):** What the agent can do.
 - **Transition Model** $P(s'|s, a)$: The probability of reaching s' if the agent takes action a in state s .
 - **Reward Function** $R(s, a, s')$: The “payoff” for a transition.

The Utility of a State

- An agent shouldn't just look for immediate rewards. It must look for **Expected Utility**.
- **Discount Factor (γ)**: Values between 0 and 1. Future rewards are worth less than immediate ones ($Utility = R_0 + \gamma R_1 + \gamma^2 R_2 \dots$).
- This models "Impatience" or uncertainty about the future.

Solving the MDP: Expected Returns & Monte Carlo

- **The Optimal Policy (π^*):** A mapping from every state to the best action.
- **Value Function (V):** The expected long-term return from a state.
- **The Bellman Equation (Exact):**

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

- **Monte Carlo Methods (Estimation):** If the agent doesn't know the exact Transition Model, it cannot use Bellman. Instead, it plays out thousands of random episodes (trajectories) to the end and averages the rewards received for each state visited.

Part 4: Training & Tools

The Reward Problem: Sparse vs. Shaped

- How we define the $R(s, a, s')$ function makes or breaks the training process.
- **Sparse Rewards:** The agent receives a signal only at the very end (e.g., +1 for winning a game, 0 for all prior moves). This causes the **Temporal Credit Assignment Problem**—the agent struggles to identify *which* specific action out of thousands caused the win.
- **Reward Shaping:** Providing dense, continuous feedback (e.g., +0.1 for moving closer to a target, -0.1 for moving away). This creates a gradient that accelerates learning by rewarding sub-goals.

Learning the Model (Training)

- **Parameter Learning:** We have the graph structure; we need to fill the CPTs using data.
 - *Maximum Likelihood Estimation (MLE):* Count how often X happened when Y was true.
- **Structure Learning:** We don't know the graph; we use algorithms (like PC or Hill-Climbing) to find the best causal links based on correlation data.

Manual vs. Algorithmic Training

- **Manual:** A human expert defines $P(\textit{Jump_Success}|\textit{Low_Battery}) = 0.4$ based on domain knowledge.
- **Algorithmic:** The agent plays 10,000 games, records every failure, and uses **Gradient Descent** or **Expectation-Maximization (EM)** to find the real probabilities.

1. **pgmpy**: Best for Bayesian Networks and exact inference.
2. **pomegranate**: High-performance probabilistic modeling.
3. **Gymnasium (OpenAI Gym)**: The standard toolkit for developing and comparing RL/MDP agents.
4. **PyMC**: For Bayesian estimation and “Learning” the probabilities from data.

Part 5: Full Example - The “Noisy Sensor” Agent

An agent must decide to **Open Door** or **Wait**.

- **Hidden State:** A person is behind the door (H) or not ($\neg H$).
- **Sensor:** A noisy motion detector (M).
- **Knowledge:** * $P(H) = 0.3$ (Prior).
 - $P(M|H) = 0.9$ (True Positive).
 - $P(M|\neg H) = 0.2$ (False Positive).

Step-by-Step Manual Computation

If the sensor triggers (M), what is $P(H|M)$?

1. **Calculate Marginal $P(M)$:**

$$P(M) = P(M|H)P(H) + P(M|\neg H)P(\neg H)$$

$$P(M) = (0.9 \times 0.3) + (0.2 \times 0.7) = 0.27 + 0.14 = 0.41$$

2. **Apply Bayes' Rule:**

$$P(H|M) = \frac{P(M|H)P(H)}{P(M)} = \frac{0.27}{0.41} \approx 0.658$$

3. **Decision:** The probability jumped from 30% to ~66%. If the agent's threshold for opening is 60%, it opens the door.

Python Implementation (using pgmpy)

```
from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# 1. Define Structure
model = BayesianNetwork([('Person', 'Sensor')])

# 2. Define Probabilities (CPDs)
cpd_person = TabularCPD(variable='Person', variable_card=2, values=[[0.8, 0.2], [0.1, 0.9]])
cpd_sensor = TabularCPD(variable='Sensor', variable_card=2,
                        values=[[0.8, 0.1], [0.2, 0.9]],
                        evidence=['Person'], evidence_card=[2])

model.add_cpds(cpd_person, cpd_sensor)

# 3. Inference
infer = VariableElimination(model)
result = infer.query(variables=['Person'], evidence={'Sensor': 1})
print(result) # Should output ~0.658 for Person=1
```

Summary

- Probabilistic agents handle the **real world**, which is noisy and incomplete.
- **Bayes Nets** factor complex problems into simple local tables.
- **MDPs** allow agents to plan over time by maximizing **Expected Utility**.
- **Python** allows us to automate the math, but understanding the **Prior** and **Likelihood** is essential for debugging AI behavior.